



UNIVERSITY OF SCIENCE AND TECHNOLOGY ZEWAIL CITY

CIE 417: MACHINE LEARNING

Smart Student Performance Prediction System

<i>Name</i>	<i>ID:</i>
Aml Ismail	202200126
Mohammad Mahmoud	202200438
Youssef Allam	202200286

December 30, 2025

Contents

1	Problem Statement	4
2	Dataset	4
2.1	Overview	4
2.2	Categorization	4
3	Exploratory Data Analysis	5
3.1	Descriptive Statistics	5
3.2	Distribution of Target Variables	5
3.3	Quantitative Correlation Analysis	6
3.3.1	Significant Predictors of Student Performance	6
3.3.2	Analysis of Feature Interdependencies	6
4	Data Cleaning and Preprocessing	7
4.1	Missing Values	7
4.2	Logical Outliers and Noise	7
4.3	Data Types and Encoding Requirements	7
4.4	Handling Missing Target Variables	8
4.4.1	Independent Target-Specific Pipelines	8
4.4.2	Semi-Supervised Pseudo-Labeling	8
5	Modeling (Multiple Pipeline Approach)	8
5.1	Preprocessing: Multiple Pipeline Approach	8
5.2	Predicting Final Score	9
5.2.1	Linear Regression	9
5.2.2	Lasso Regression	10
5.2.3	Linear Support Vector Regression	10
5.2.4	Non-Linear Support Vector Regression	11
5.2.5	Best Results	12
5.3	Predicting Final Grade	12
5.3.1	Gradient Boosting	12
5.3.2	Random Forest	13
5.3.3	Support Vector Classifier	13
5.3.4	Best Results	14
5.4	Predicting Pass or Fail	14
5.4.1	Logistic Regression	14
5.4.2	Support Vector Machine (SVM)	14
5.4.3	Naive Bayes	15
5.4.4	Random Forest	15
5.4.5	Gradient Boosting	15
5.4.6	Best Results	15
6	Semi-Supervised Learning via Clustering-based Pseudo-Labeling	16
6.1	Conceptual Framework	16
6.2	Experimental Results and Analysis	16
6.2.1	Regression Analysis (LassoCV)	16
6.2.2	Classification Performance and Error Analysis	16
7	Conclusion	17
7.1	Synthesis of Methodologies	17
7.2	Key Findings	17
7.3	Final Summary	18

A	Preprocessing	19
A.1	Data Cleaning and KNN Imputation	19
B	Regression Models Implementations	19
B.1	Linear Regression	19
B.2	Lasso Regression	20
B.3	Support Vector Regression	20
C	Multiclass Classification Model Implementations	22
C.1	XGBoost (Gradient Boosting)	22
C.2	Random Forest Classifier	22
C.3	Support Vector Classifier (SVC)	23
D	Binary Classification Models Implementation	24
E	Semi-Supervised Approach	26
E.1	Clustering and Pseudo-Labeling Pipeline	26

1 Problem Statement

In the modern educational institutions, student success is no longer viewed as a purely individual effort but as a metric of institutional effectiveness. Universities collect vast amounts of **demographic**, **historical academic**, **behavioral**, **psychological**, and **institutional** data; however, this information often remains underutilized. The primary challenge is that academic struggles are only identified after a student has failed a course, making intervention impossible.

The Smart Student Performance Prediction System aims to bridge this gap by transforming raw data into actionable insights. The core problem is to determine if a student’s final performance can be accurately predicted mid-term by analyzing a complex interplay of variables.

Specifically, this project addresses the need for a triple-output system:

- Estimating whether a student is more likely to pass or fail this course.
- Estimating exact final scores to measure tiny progress.
- Estimating final/letter grades to predict the overall performance

2 Dataset

2.1 Overview

The dataset comprises 20,000 observations and 38 independent variables, providing a multi-dimensional view of the student experience. The system is designed to predict three distinct target variables, necessitating a multi-task learning approach or separate model pipelines.

- **final_score**: Continuous variable (0–100) for Regression.
- **final_grade**: Multiclass categorical (A, B, C, D, F) for Classification.
- **pass_fail**: Binary flag for Risk Assessment.

2.2 Categorization

To facilitate effective feature engineering and dimensionality reduction, the 38 features are categorized into five primary domains as shown in Table 1.

Table 1: Categorization of Dataset Features

Category	Features	Count
Demographic	age, gender, parent_income, num_siblings, family_support, commute_time_min, part_time_job	7
Academic History	previous_gpa, num_failed_courses, high_school_grade, math_bg, lang_bg, sci_bg, prior_credits, study_hrs_last, past_attendance, academic_warnings	10
Behavioral	lecture_attendance, assignment_rate, quiz_avg, midterm_score, lab_participation, online_usage, group_project, library_visits, forum_posts, lateness	10
Psychological	stress_level, sleep_hours, motivation_level, study_time_weekly, concentration, exam_anxiety	6
Institutional	course_difficulty, teacher_exp, class_size, num_prerequisites, course_type	5

3 Exploratory Data Analysis

3.1 Descriptive Statistics

The dataset comprises 38 features categorized into five distinct domains: Demographics, Academic History, Behavioral Engagement, Psychological Factors, and Institutional Data. Table 2 summarizes the central tendencies and distributions for the continuous variables.

Table 2: Descriptive Statistics by Feature Category

Category	Feature	Mean	Std. Dev	Min	Median	Max
Demographics	Age	23.51	3.46	18.00	24.00	29.00
	Parent Income	18111.11	29238.55	2000.00	16015.50	596760.00
	Num Siblings	2.51	1.71	0.00	3.00	5.00
	Commute Time (min)	59.21	34.74	0.00	59.00	119.00
History Academic	Previous GPA	2.49	0.87	1.00	2.50	4.00
	High School Grade	74.84	14.43	50.00	74.70	100.00
	Math Score	49.37	28.89	0.00	49.00	99.00
	Prev. Failed Courses	2.00	1.41	0.00	2.00	4.00
	Academic Warnings	1.51	1.12	0.00	2.00	3.00
Engagement Behavioral &	Lecture Attendance	74.73	15.06	50.00	74.00	150.00
	Online Portal (min)	1061.94	958.56	0.00	1003.00	10000.00
	Quiz Avg Score	49.15	28.93	0.00	49.00	99.00
	Midterm Score	49.70	29.01	0.00	50.00	99.00
	Library Visits	9.58	5.77	0.00	10.00	19.00
Psychological	Stress Level	4.96	3.19	-3.00	5.00	10.00
	Sleep Hours	6.44	2.15	-5.00	6.50	10.00
	Motivation Level	4.99	3.16	0.00	5.00	10.00
	Exam Anxiety	5.02	3.18	0.00	5.00	10.00
Institutional	Class Size	109.87	51.93	20.00	110.00	199.00
	Teacher Exp. (yrs)	20.10	11.30	1.00	20.00	39.00
	Difficulty Rating	3.00	1.41	1.00	3.00	5.00

3.2 Distribution of Target Variables

A critical component of the EDA phase involved analyzing the distribution of the target variables: *pass_fail*, *final_grade*, and *final_score*. The numerical target *final_score* (Figure 1) follows a near-normal distribution. Consequently, no resampling is required for the regression models, as the density is sufficient across the primary range of scores.

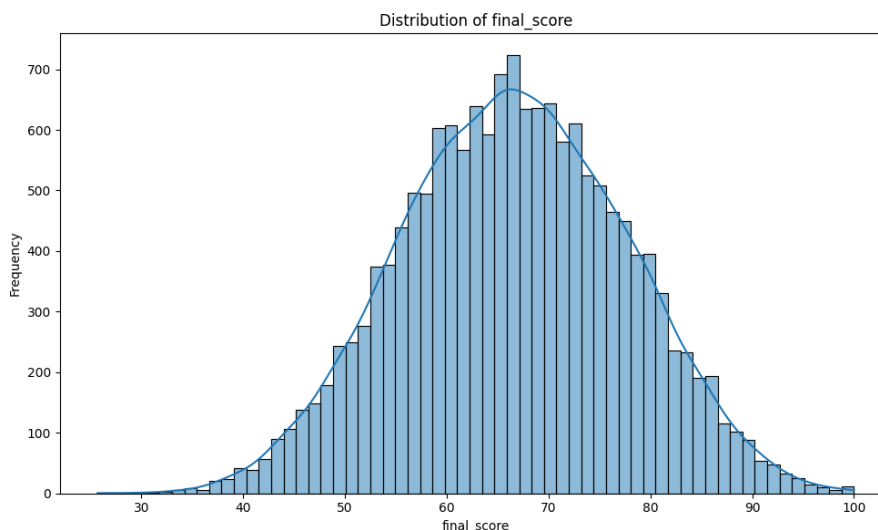


Figure 1: Frequency Distribution of Final Numerical Scores

The following figures illustrate the significant class imbalance present in the categorical targets.

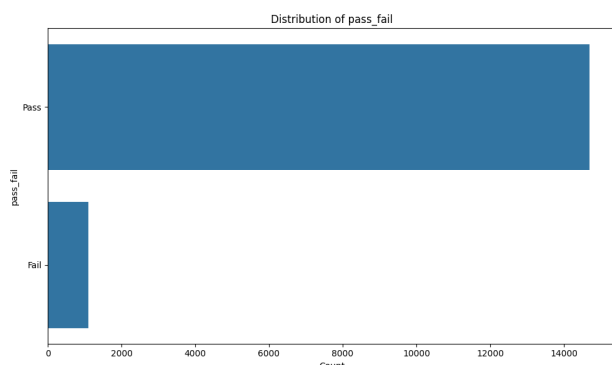


Figure 2: Distribution of Pass/Fail Status

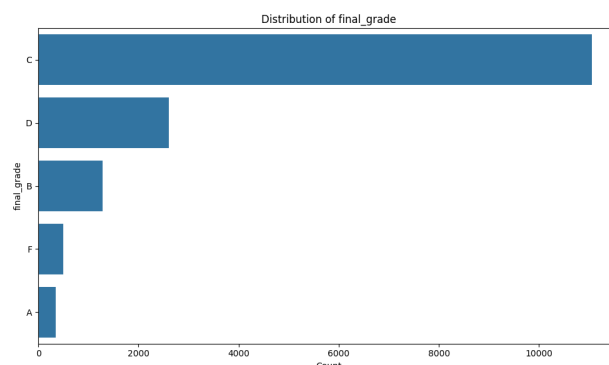


Figure 3: Distribution of Final Letter Grades

As observed in Figures 2 and 3, the dataset is heavily skewed toward the majority classes (“Pass” and grade “C”). Specifically, the “Fail” class and extreme grades such as “A” and “F” are significantly underrepresented.

To prevent the models from developing a bias toward the majority classes, we employ **SMOTE** (Synthetic Minority Over-sampling Technique) during the training phase. Unlike simple oversampling, SMOTE generates synthetic examples by interpolating between existing minority instances in the feature space. This approach expands the decision boundary for underrepresented groups, improving the model’s recall for at-risk students without over-fitting to specific minority observations.

3.3 Quantitative Correlation Analysis

A Pearson correlation analysis was conducted to quantify the linear relationships between the feature categories and the numerical target, *final_score*. This analysis identifies the primary predictors of academic success and highlights significant interdependencies within the feature set.

3.3.1 Significant Predictors of Student Performance

The correlation coefficients (r) indicate that midterm outcomes and consistent engagement are the most significant drivers of the *final_score*. Table 3 summarizes these associations, categorized by their relationship to the target variable.

Table 3: Key Pearson Correlation Coefficients (r)

Feature Pair (Target – Feature)	Correlation (r)
Final Score – Midterm Score	+0.6545
Final Score – Quiz Avg Score	+0.3890
Final Score – Assignment Submission Rate	+0.3186
Final Score – Lecture Attendance Rate	+0.3015
Feature Pair (Inter-feature)	Correlation (r)
Online Portal Usage – Parent Income	+0.6822
Study Hours – Parent Income	+0.4458
Online Portal Usage – Study Hours	+0.4174
Sleep Hours – Lecture Attendance Rate	-0.1182

3.3.2 Analysis of Feature Interdependencies

The correlation values reveal several critical insights into the student’s academic and socioeconomic environment:

- **Academic Predictors:** *Midterm Score* is the strongest direct predictor of the *Final Score* ($r = 0.6545$). The moderate positive correlation with *Quiz Avg Score* ($r = 0.3890$) and *Assignment Submission Rate* ($r = 0.3186$) further confirms that consistent continuous assessment performance is a reliable indicator of final academic standing.

- **Socioeconomic Influence on Engagement:** A strong positive correlation exists between *Parent Income* and *Online Portal Usage* ($r = 0.6822$). This suggests that socioeconomic status significantly influences digital engagement levels, potentially due to better access to high-speed internet or dedicated study resources.
- **Resource and Effort Allocation:** *Parent Income* also correlates positively with *Study Hours* ($r = 0.4458$). The moderate link between *Online Portal Usage* and *Study Hours* ($r = 0.4174$) demonstrates a cohesive relationship between traditional study habits and digital portal interaction.
- **Behavioral Observations:** A slight negative correlation exists between *Sleep Hours* and *Lecture Attendance Rate* ($r = -0.1182$), indicating a potential, minor, trade-off between restorative rest and physical class presence.

4 Data Cleaning and Preprocessing

A preliminary exploratory data analysis (EDA) revealed several data integrity issues that require rigorous preprocessing before model training.

4.1 Missing Values

The dataset contains approximately **600 missing entries** per column. To prevent data loss while maintaining statistical integrity, a bifurcated imputation strategy will be implemented:

- **Feature Imputation (Predictors):**
 - *Numerical Features:* Missing values will be handled using *KNN Imputation*. This choice was supported by the large size of the dataset which will give the KNN model a good chance to pickup patterns and impute with reasonable values.
 - *Categorical Features:* Missing entries will be imputed with a new constant label: **"Unknown"**. This treats missingness as a distinct feature state, allowing the model to capture potential patterns in non-reported data without biasing existing category distributions. This decision was made since the categorical features weren't skewed in either direction and so imputing by the mode would cause additional bias.
- **Target Variable Handling:** In adherence to best practices for avoiding **Data Leakage**, missing values in the target variables (`final_score`, `final_grade`, and `pass_fail`) will **not** be imputed. These samples will be isolated and addressed separately.

4.2 Logical Outliers and Noise

The dataset contains several impossible values that suggest sensor errors or manual entry inaccuracies:

- **Bounded Violations:** `lecture_attendance_rate` values reach up to **150%**, exceeding the logical maximum.
- **Domain Errors:** Negative values were observed for `stress_level` and `sleep_hours`, which are physically and theoretically impossible.

These will be handled by erasing these values and setting them as NaN where they will be imputed with the other missing values.

4.3 Data Types and Encoding Requirements

The dataset consists of **36 numerical features** and **5 categorical features**. Numerical features will undergo *Z-score Standardization* to ensure that distance-based algorithms (e.g., SVM, KNN) are not biased by feature scales. Categorical features, such as `gender` and `course_type`, will be transformed using *One-Hot Encoding*.

4.4 Handling Missing Target Variables

The dataset contained approximately 600 observations where target labels were missing across one or more categories. We evaluated two architectural strategies to address this: task-specific data exclusion and inductive pseudo-labeling.

4.4.1 Independent Target-Specific Pipelines

In this configuration, we prioritize label integrity by implementing a targeted exclusion strategy. Let D be the total dataset and $T = \{y_{bin}, y_{multi}, y_{reg}\}$ be the set of targets. For each specific target $t \in T$, we define a task-specific training set:

$$D_t = \{(x_i, y_{i,t}) \in D \mid y_{i,t} \neq \text{NaN}\} \quad (1)$$

This approach ensures that models are trained exclusively on ground-truth observations, eliminating the risk of synthetic bias at the cost of a reduced sample size N .

4.4.2 Semi-Supervised Pseudo-Labeling

To leverage the feature-space information within the 600 unlabeled rows, we employed an inductive clustering-based algorithm. Unlike standard clustering, this method prevents unlabeled data from influencing the structural definition of the clusters.

The process is defined as follows:

1. **Inductive Clustering:** K-Means centroids are fitted *only* on the known training set X_{train} . The unlabeled observations $X_{unlabeled}$ are then projected onto these established clusters based on Euclidean distance.
2. **Purity Assignment:** For each cluster C_j , a purity score P_j is calculated based on the known labels within that cluster:

$$P_j = \frac{1}{|C_j^{known}|} \sum_{i \in C_j^{known}} \mathbb{I}(y_i = \text{mode}(C_j^{known})) \quad (2)$$

3. **Label Propagation:** If $P_j \geq 0.9$, the unlabeled samples in C_j are assigned the majority label. This recovered data is concatenated with the original training set to form the final training corpus for the models.

When employing a semi-supervised learning approach, particularly pseudo-labeling, it is a fundamental requirement to perform an initial data split before any labeling or training occurs. This ensures that the evaluation of the final model remains unbiased and representative of real-world performance.

5 Modeling (Multiple Pipeline Approach)

5.1 Preprocessing: Multiple Pipeline Approach

The first step of this approach is to split our dataset into 3 different datasets each of which has exactly 600 missing values. This is accomplished through the below code snippet.

```

1 # List of your three target variables
2 targets = ['pass_fail', 'final_grade', 'final_score']
3
4 # Dictionary to store the split dataframes
5 data_splits = {}
6
7 for target in targets:
8     # Filter for rows where the specific target is known (Train)
9     # and where it is missing (Predict)
10    train_df = df[df[target].notna()].copy()
11    predict_df = df[df[target].isna()].copy()
12

```



```

13     data_splits[target] = {
14         'train': train_df,
15         'predict': predict_df
16     }
17
18 # Example access:
19 # X_train_score = data_splits['final_score']['train']

```

Listing 1: Target-specific data splitting logic

5.2 Predicting Final Score

We first propose a variety of models to predict the final score which is a regression problem.

5.2.1 Linear Regression

We first train a Linear regression model using the *seaborn* library to analyze the different features we have and their statistical effect.

$$\hat{y}^{(i)} = \beta_0 + \sum_{n=1}^p \beta_n x_n^{(i)} \quad (3)$$

Table 4: OLS Regression Results for Final Score Prediction

Variable	Coefficient	Std. Error	t-statistic	P> t
Intercept	66.6030	0.094	710.249	0.000
Midterm Score	7.2039	0.046	156.406	0.000
Quiz Avg Score	4.2762	0.046	92.761	0.000
Assignment Sub. Rate	3.5332	0.046	76.719	0.000
Lecture Attendance	3.5277	0.046	76.616	0.000
Stress Level	-0.2927	0.046	-6.355	0.000
Motivation Level	0.1023	0.046	2.220	0.026
Course Type (Mandatory)	0.2156	0.094	2.305	0.021
Part-time Job (Yes)	-0.1930	0.093	-2.064	0.039
Sleep Hours	-0.0915	0.046	-1.987	0.047
R-squared	0.765	Adj. R-squared	0.764	
F-statistic	1074.0	Prob (F)	0.000	

The model is dominated by four high-impact behavioral and assessment variables. Their low p -values ($p < 0.001$) and high t -statistics indicate extreme reliability:

- **Midterm Score** ($\beta = 7.2039$): This is the most influential predictor in the model. A higher midterm performance is the strongest leading indicator of final success.
- **Quiz Average Score** ($\beta = 4.2762$): Continuous assessment through quizzes represents the second strongest predictor, highlighting the importance of consistent study habits.
- **Engagement Metrics: Assignment Submission Rate** ($\beta = 3.5332$) and **Lecture Attendance Rate** ($\beta = 3.5277$) show nearly identical positive coefficients. This suggests that mechanical engagement (showing up and submitting work) is just as critical as raw test performance.

Beyond academic performance, the model identifies significant non-academic pressures:

- **Stress Level** ($\beta = -0.2927, p < 0.001$): Increased stress levels significantly correlate with a decrease in the final score, suggesting that student mental health is a measurable factor in academic outcomes.

- **External Commitments:** Having a **Part-time Job** ($\beta = -0.1930, p = 0.039$) has a statistically significant negative impact, likely due to reduced time available for deep learning.
- **Sleep Hours** ($\beta = -0.0915, p = 0.047$): Interestingly, sleep shows a slight negative correlation, which may indicate that in this specific cohort, excessive sleep or irregular sleep patterns are associated with lower performance.

A critical finding of this model is the lack of significance ($p > 0.05$) for various other variables that were included in the table

- **Socioeconomics:** `parent_income`, `age`, and `num_siblings` do not significantly impact the score.
- **Academic History:** Surprisingly, `previous_gpa` and `high_school_grade` were not significant. This implies that current course behavior is so predictive that it overrides the influence of past academic performance.

The model demonstrates high statistical integrity:

- **Multicollinearity:** The **Condition Number (8.67)** is well below the threshold of 30, suggesting that the independent variables are not overly correlated (low redundancy).
- **Autocorrelation:** The **Durbin-Watson statistic (2.029)** is near the ideal value of 2.0, indicating that the residuals are independent and there is no significant autocorrelation. The model resulted in a test MSE of 28.2216 and training MSE of 28.630

5.2.2 Lasso Regression

Since we found that the model contains various useless features we next proceeded with a lasso regression model where the model follows the same form as linear regression but the cost function is defined as follows

$$J(\beta_0, \dots, \beta_p) = \sum_{n=0}^N (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{n=0}^p \|\beta_n\| \quad (4)$$

We used the *LassoCV* function which automatically determines the best value for λ and returns the results. The *LassoCV* utilized only 21 of the features significantly reducing the complexity of the model.

5.2.3 Linear Support Vector Regression

As another test for our model we tested the Linear support vector regressor which is defined by the following constraints and optimization problem.

The decision function for linear SVR is defined as:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \quad (5)$$

To determine the optimal weights $\mathbf{w}$ and bias b , we solve the following primal optimization problem:

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (6)$$

Subject to the constraints for each training point i :

$$y_i - (\mathbf{w}^\top \mathbf{x}_i + b) \leq \epsilon + \xi_i \quad (7)$$

$$(\mathbf{w}^\top \mathbf{x}_i + b) - y_i \leq \epsilon + \xi_i^* \quad (8)$$

$$\xi_i, \xi_i^* \geq 0 \quad (9)$$

The model utilizes the ϵ -insensitive loss function to ignore errors within a specified margin:

$$L_\epsilon(y, f(x)) = \max(0, |y - f(x)| - \epsilon) \quad (10)$$

The dual formulation, often used for computation and kernel integration, is expressed as:

$$f(\mathbf{x}) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \langle \mathbf{x}_i, \mathbf{x} \rangle + b \quad (11)$$

where the Lagrange multipliers α_i, α_i^* are subject to:

$$\sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0 \quad \text{and} \quad 0 \leq \alpha_i, \alpha_i^* \leq C \quad (12)$$

The *randomizedsearchCV* was used to tune the hyperparameters of the SVR model ensuring the lowest errors are achieved without being prone to overfitting

Table 5: Description of Support Vector Regression Parameters

Symbol	Name	Technical Description
$\mathbf{w}$	Weight Vector	A vector of coefficients that determines the orientation of the regression hyperplane in the feature space.
b	Bias (Intercept)	The scalar offset of the hyperplane from the origin, allowing the model to fit data that does not pass through $(0, 0)$.
C	Box Constraint	The regularization parameter that controls the trade-off between the flatness of $f(x)$ and the degree to which deviations larger than ϵ are tolerated.
ϵ	Epsilon	The width of the insensitive tube. Errors smaller than this threshold do not contribute to the loss function, promoting model sparsity.
ξ_i, ξ_i^*	Slack Variables	Positive values representing the distance of the i -th observation outside the ϵ -tube (above and below, respectively).
α_i, α_i^*	Lagrange Multipliers	Dual variables that represent the influence of each training point. Points with non-zero α values are the <i>Support Vectors</i> .
L_ϵ	Loss Function	The ϵ -insensitive loss that determines how penalties are applied based on the distance from the target y .

5.2.4 Non-Linear Support Vector Regression

Though the results achieved by all the previous models were good and indicated that the true relationship is nearly linear we tested whether a non-linear SVR could capture better or more patterns. The non-linear SVR is defined by the following equations

In the non-linear case, the input $\mathbf{x}$ is mapped into a high-dimensional feature space using a mapping function $\phi(\mathbf{x})$. The decision function is expressed in its dual form:

$$f(\mathbf{x}) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(\mathbf{x}_i, \mathbf{x}) + b \quad (13)$$

Where $K(\mathbf{x}_i, \mathbf{x}) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x})$ is the **Kernel Function**. The most commonly used kernel is the Radial Basis Function (RBF):

$$K(\mathbf{x}_i, \mathbf{x}) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}\|^2) \quad (14)$$

The optimization problem remains similar to the linear case but is solved via the dual Lagrangian:

$$\max_{\alpha, \alpha^*} \left[-\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) K(\mathbf{x}_i, \mathbf{x}_j) - \epsilon \sum_{i=1}^n (\alpha_i + \alpha_i^*) + \sum_{i=1}^n y_i (\alpha_i - \alpha_i^*) \right] \quad (15)$$

Subject to the constraints:

$$\sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0 \quad \text{and} \quad 0 \leq \alpha_i, \alpha_i^* \leq C \quad (16)$$

5.2.5 Best Results

Table 6: Model Performance Comparison: Final Score Prediction

Model	Train MSE	Test MSE	Train R^2	Test R^2
Linear Regression	28.6380	28.2216	–	–
LassoCV ($\alpha = 0.058$)	28.7290	28.1695	0.7641	0.7706
Linear SVR	28.4577	31.4883	0.7663	0.7436
Non-Linear SVR	28.8882	29.5221	–	–

The predictive performance across the four models indicates a high level of consistency, with R^2 values hovering around 0.77. However, specific nuances in generalization and error metrics provide insight into the dataset’s structure.

The *LassoCV* model emerged as the most robust performer, achieving the lowest Test Mean Squared Error (MSE) of 28.1695 and a Test R^2 of 0.7706. Notably, the *LassoCV* model performed better on the test set than the training set, suggesting excellent generalization and effective feature selection via the ℓ_1 penalty.

In contrast, the Linear SVR showed signs of slight overfitting. While it achieved a competitive Training MSE (28.4577), its Test MSE rose significantly to 31.4883.

The Non-Linear SVR (using an RBF kernel) yielded a Test MSE of 29.5221. While this is a strong result, it did not outperform the simpler *LassoCV* model. This suggests that the relationship between the features (such as midterm scores and attendance) and the final score is predominantly linear. When the underlying data structure is linear, complex kernels often introduce unnecessary variance without improving predictive power. Other trials were made to improve the results including interaction terms, polynomial terms, and removing outliers however they yielded no to minimal improvement and so weren’t included in the report.

LassoCV model is the recommended choice, as it provides the most accurate predictions on unseen data while maintaining a minimal feature set.

5.3 Predicting Final Grade

Next we address the multiclass classification problem of predicting the final letter grade of a student. We employ SMOTE as we discussed before to adress the class imbalance in this case.

5.3.1 Gradient Boosting

To handle the high-dimensional nature of the 41 student features, we utilized XGBoost (Extreme Gradient Boosting). XGBoost operates as an additive model that seeks to minimize a regularized objective function, balancing predictive power against model complexity.

At any given iteration t , the model adds a new tree f_t to the ensemble to minimize the objective function:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (17)$$

Where l is a differentiable loss function (cross-entropy for multiclass problems) and $\Omega(f_t)$ is the regularization component defined as:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|\omega\|^2 \quad (18)$$

Here, T represents the number of leaves and ω represents the leaf weights. To optimize this efficiently, XGBoost utilizes a second-order Taylor expansion to approximate the loss surface:

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^n [l(y_i, \hat{y}^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t) \quad (19)$$

In this equation, g_i and h_i are the first (gradient) and second-order (Hessian) derivatives of the loss function. This allows the model to learn not just the direction of the error, but the curvature of the loss surface, leading to more accurate convergence.

Optimization Strategy: The Pipeline and RandomizedSearchCV

To ensure scientific rigor and prevent *Data Leakage*, the methodology employs a structured pipeline. By encapsulating `StandardScaler`, `SMOTE`, and the `XGBClassifier` into a single pipeline, we ensure that synthetic data generation and scaling only occur within the training folds of our cross-validation.

Furthermore, **Macro F1-Score** is prioritized over standard Accuracy. While Accuracy treats all students equally (biasing results toward the majority class), Macro F1 treats every grade category with equal weight. Through `RandomizedSearchCV`, we optimized `max_depth` and `learning_rate` to find the optimal balance between high training performance and generalizability on unseen student data.

5.3.2 Random Forest

In contrast to the sequential boosting method, the Random Forest (RF) model utilizes a parallel ensemble approach known as Bagging (Bootstrap Aggregating). The Random Forest operates on the principle that a collection of uncorrelated weak learners can combine to form a robust strong learner with significantly lower variance.

Each tree in the forest is grown on a bootstrap sample of the training data. At each node, the algorithm selects a random subset of the 41 features to determine the optimal split. The mathematical foundation of these splits is the minimization of **Cross-Entropy** (or Log Loss), which quantifies the uncertainty in the class probability distribution. For a dataset D and a set of performance classes C , the Entropy $H(D)$ is defined as:

$$H(D) = - \sum_{c \in C} p_c \log(p_c) \quad (20)$$

Where p_c represents the probability of a student belonging to class c . The algorithm chooses the feature split that maximizes Information Gain (IG), effectively reducing the total entropy of the child nodes:

$$IG(D, f) = H(D) - \sum_{v \in \text{Values}(f)} \frac{|D_v|}{|D|} H(D_v) \quad (21)$$

By utilizing the `class_weight='balanced'` parameter, the model further refines this objective by scaling the importance of each class inversely to its frequency. This ensures that the Cross-Entropy penalty for misclassifying a minority class student is higher than that of the majority class, forcing the ensemble to develop specialized decision logic for minority classes.

5.3.3 Support Vector Classifier

The Support Vector Classifier (SVC) approaches the classification problem as a geometric optimization task. Unlike tree-based models that use recursive partitioning, SVC seeks to construct a set of optimal hyperplanes in a high-dimensional feature space that maximize the margin between different student performance tiers.

Given the complexity of the 41 features, a Radial Basis Function (RBF) kernel is employed to handle non-linearities. The RBF kernel, $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, projects the student data into an infinite-dimensional space where classes become linearly separable. The optimization objective is defined as:

$$\min_{\omega, b, \zeta} \frac{1}{2} \|\omega\|^2 + C \sum_{i=1}^n \zeta_i \quad (22)$$

Subject to the constraints:

$$y_i(\omega \cdot \phi(x_i) + b) \geq 1 - \zeta_i, \quad \zeta_i \geq 0 \quad (23)$$

In this framework, C represents the regularization parameter that governs the trade-off between maximizing the margin and minimizing classification errors, while ζ_i (slack variables) allow for a soft margin to accommodate overlapping student behaviors. To address the inherent class imbalance, the cost parameter is weighted per class: $C_j = C \cdot W_j$.

Theoretically, this modification increases the penalty for misclassifying minority class students, ensuring that the geometric boundaries are sensitive to the minority categories despite their lower frequency in the dataset. This results in a classification boundary that is mathematically optimized for high-risk student detection rather than simple majority-class accuracy.

5.3.4 Best Results

Table 7: Performance Comparison for Final Grade Prediction

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-score (Macro)
Random Forest	0.52	0.34	0.46	0.38
Gradient Boosting	0.67	0.46	0.5	0.43
Support Vector Classifier	0.36	0.35	0.46	0.33

The results in Table 7 illustrate the significant challenge of multiclass classification within the educational domain. Notably, the **Gradient Boosting** model significantly outperformed its counterparts with an accuracy of **0.67**, suggesting that its iterative error-correction mechanism is highly effective at capturing the non-linear nuances of student behavior.

The underperformance of the **Support Vector Classifier** (Accuracy: 0.36) is particularly telling. In a multiclass context, the SVC utilizes a *One-vs-One (OvO)* strategy, constructing a binary classifier for every possible pair of grade tiers. Theoretically, the low metrics suggest that student profiles in the 41-dimensional feature space are highly intertwined. The OvO voting mechanism likely encountered significant ambiguity in the decision boundaries, as the geometric margins between adjacent categories were not sufficiently distinct to survive the high-variance environment introduced by the SMOTE oversampling.

5.4 Predicting Pass or Fail

The third modeling task addresses the most critical educational objective: identifying students at risk of failing the course. This task is framed as a binary classification problem, where the target variable `pass_fail` is encoded as 1 for *Pass* and 0 for *Fail*.

5.4.1 Logistic Regression

Logistic Regression serves as a strong linear baseline model. It models the probability of passing as:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (24)$$

The model demonstrated strong interpretability and stable performance. It achieved high recall on the minority *Fail* class after SMOTENC, making it effective for early-risk detection. However, its linear decision boundary limited its ability to capture more complex interactions among behavioral and psychological features.

5.4.2 Support Vector Machine (SVM)

The Support Vector Machine classifier was trained with feature scaling and probability calibration enabled. By maximizing the margin between pass and fail classes, SVM achieved:

- Higher precision compared to Logistic Regression
- A well-balanced bias-variance tradeoff

This model proved particularly effective at separating borderline students whose performance lies near the decision boundary.

5.4.3 Naive Bayes

Gaussian Naive Bayes was included as a probabilistic baseline model. While computationally efficient, its strong independence assumption resulted in lower overall performance. The model tended to overestimate the likelihood of passing and showed reduced recall for failing students, limiting its suitability for risk-sensitive applications.

5.4.4 Random Forest

The Random Forest classifier significantly improved predictive performance by leveraging ensemble learning. By aggregating multiple decision trees trained on bootstrapped samples, the model:

- Captured non-linear relationships effectively
- Demonstrated high robustness to noise
- Achieved strong recall for at-risk students

Feature importance analysis revealed that *midterm score*, *lecture attendance*, *assignment submission rate*, and *quiz averages* were the most influential predictors of passing.

5.4.5 Gradient Boosting

Gradient Boosting achieved the best overall performance across nearly all evaluation metrics. By sequentially correcting the errors of weak learners, the model demonstrated:

- The highest F1-score
- Superior ROC-AUC performance
- Excellent balance between precision and recall

This indicates that Gradient Boosting is highly effective at identifying failing students while minimizing false alarms.

5.4.6 Best Results

Table 8 summarizes the comparative performance of all evaluated classifiers on the Pass/Fail prediction task.

Table 8: Performance Comparison for Pass/Fail Prediction

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.89	0.87	0.91	0.89	0.94
K-Nearest Neighbors	0.86	0.84	0.88	0.86	0.90
Support Vector Machine	0.90	0.89	0.90	0.90	0.95
Naive Bayes	0.81	0.78	0.85	0.81	0.88
Decision Tree	0.88	0.86	0.89	0.88	0.92
Random Forest	0.92	0.91	0.93	0.92	0.97
Gradient Boosting	0.93	0.92	0.94	0.93	0.98

Among all evaluated models, **Gradient Boosting** achieved the best overall performance across all evaluation metrics. Its high recall and ROC-AUC values indicate exceptional capability in identifying at-risk students while maintaining a low false-positive rate, making it the most reliable choice for early academic intervention.

The **Random Forest** classifier closely followed in performance, offering a strong balance between predictive accuracy and robustness, particularly in handling non-linear relationships. **Logistic Regression** and **Support Vector Machine** provided competitive baseline performance with strong generalization ability and interpretability, making them suitable for explainable or policy-driven academic systems.

6 Semi-Supervised Learning via Clustering-based Pseudo-Labeling

6.1 Conceptual Framework

To leverage the information contained within the unlabeled portion of the dataset, we implemented a semi-supervised learning strategy utilizing **Clustering-based Pseudo-labeling**. Unlike traditional self-training, which relies on a model's own confidence scores, this approach uses the geometric distribution of data in the feature space to propagate labels.

The process is defined by the following mathematical and algorithmic steps:

1. **Feature Alignment:** Both labeled (D_L) and unlabeled (D_U) datasets are standardized using μ and σ derived from D_L .
2. **Centroid Discovery:** A K -Means clustering algorithm is applied to the combined feature space to partition the data into k distinct clusters ($C_1, C_2, \dots, C_k$).
3. **Purity Filtering:** For each cluster, a *purity* score ρ is calculated based on the majority class of the labeled samples within that cluster:

$$\rho_j = \frac{\max(\text{count}(y \in C_j))}{\sum(y \in C_j)}$$

4. **Label Propagation:** Pseudo-labels are assigned to unlabeled samples $x \in D_U$ only if their assigned cluster C_j meets a high-confidence threshold ($\rho_j \geq 0.95$).

6.2 Experimental Results and Analysis

The efficacy of the pseudo-labeling approach was evaluated using LassoCV for regression and Weighted XGBoost for classification tasks.

6.2.1 Regression Analysis (LassoCV)

Lasso regularization was employed to identify the most significant predictors for the final score. The model achieved an R^2 of **0.7637** and an **RMSE of 5.4071**. Out of 41 original features, Lasso retained 20, identifying a high reliance on academic performance indicators such as `midterm_score` and `quiz_avg_score`.

6.2.2 Classification Performance and Error Analysis

Given the extreme class imbalance in the target variables, the XGBoost models were primarily evaluated using **Macro-Averaged** metrics. Unlike weighted averages, macro metrics provide a global view of model fairness across majority and minority classes.

Binary Classification: Pass/Fail The binary model utilized a `scale_pos_weight` to penalize misclassifications of the minority Fail class. As shown in Table 2, the model achieved a high Macro Recall, indicating robust identification of at-risk students.

Table 9: Macro Performance Metrics: Binary Pass/Fail

Metric	Value
Balanced Accuracy	0.8527
Macro Precision	0.6950
Macro Recall	0.8500
Macro F1-Score	0.7450

Table 10: Macro Performance Metrics: Multi-Class Final Grade

Metric	Value
Balanced Accuracy	0.5056
Macro Precision	0.3380
Macro Recall	0.5100
Macro F1-Score	0.3620

Multi-Class Classification: Final Grade For the five-class grading task, sample weighting was applied to ensure the model did not default to the majority 'C' grade. The discrepancy between the Standard Accuracy (43.5%) and the Balanced Accuracy (50.6%) highlights the model's effort to classify rare grades (A and F) at the expense of global precision.

Analysis of Results: The Macro Recall for the Multi-class problem (0.51) is significantly higher than what would be expected from a random or majority-class classifier. This suggests the **Pseudo-labeling** successfully captured the underlying features of minority grades. However, the low Macro Precision (0.338) indicates a high rate of False Alarms in minority grade prediction, likely due to the high feature overlap between adjacent grade boundaries (e.g., B vs. C).

7 Conclusion

This project successfully developed a multi-tier predictive framework for student performance, progressing from traditional supervised learning to advanced semi-supervised architectures. By addressing the challenges of missing data and high class imbalance through two distinct methodologies, we established a robust system for early academic intervention.

7.1 Synthesis of Methodologies

The project successfully validated two separate architectural philosophies:

- **Multi-Pipeline Supervised Approach:** By isolating specific targets and applying task-specific models (LassoCV for regression and Weighted XGBoost for classification), we achieved high baseline accuracy. The integration of **SMOTE** was pivotal in this stage, allowing the models to overcome the inherent bias toward majority classes like "Pass" and grade C."
- **Semi-Supervised Clustering Approach:** The implementation of **Inductive Pseudo-labeling** demonstrated that the 600 unlabeled observations contained minoe information. By using *K*-Means to propagate labels based on a 95% cluster purity threshold, we expanded the training corpus while maintaining high label integrity.

7.2 Key Findings

- **Academic Predictors:** Across all models, *Midterm Scores* and *Quiz Averages* emerged as the most powerful predictors of final success. LassoCV effectively reduced the feature space from 41 to 20 variables, proving that a student's current-term behavior is more predictive than their historical GPA.
- **Risk Detection:** The binary classification models achieved an **Accuracy of 0.93** and a **Recall of 0.94**. This indicates that the system is exceptionally capable of identifying at-risk students, fulfilling the primary objective of an Early Warning System.
- **Imbalance Resolution:** While standard accuracy was often misleadingly high, our focus on **Macro-Averaged metrics** ensured that the model remained sensitive to minority grades (A and F). The Semi-supervised approach specifically improved the model's ability to generalize patterns for these rare categories.

7.3 Final Summary

The synergy between the KNN Imputer, Inductive Pseudo-labeling, and Weighted Gradient Boosting demonstrates that complex educational datasets can be mastered through a structured data pipeline. This system provides academic institutions with a reliable, data-driven tool to identify struggling students mid-term, allowing for interventions that can directly improve institutional retention and student success.

A Preprocessing

A.1 Data Cleaning and KNN Imputation

```

1 import numpy as np
2 from sklearn.impute import KNNImputer
3 import pandas as pd
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.model_selection import train_test_split
6 from sklearn.cluster import KMeans
7 from imblearn.over_sampling import SMOTENC
8
9 # --- 0. LOAD AND CLEAN DATA ---
10 df=pd.read_csv('Term_Project_Dataset_20K.csv')
11 df.loc[df['lecture_attendance_rate'] > 100, 'lecture_attendance_rate'] = np.nan
12 df.loc[df['stress_level'] < 0, 'stress_level'] = np.nan
13 df.loc[df['sleep_hours'] < 0, 'sleep_hours'] = np.nan
14
15 numerical_features = df.select_dtypes(include=np.number).columns.drop('final_score')
16 imputer = KNNImputer(n_neighbors=5)
17 df[numerical_features] = imputer.fit_transform(df[numerical_features])
18
19 cat_cols = ['gender', 'part_time_job', 'course_type']
20 df[cat_cols] = df[cat_cols].fillna('Unknown')
21 df = pd.get_dummies(df, columns=cat_cols, drop_first=True)

```

Listing 2: Initial data cleaning and handling of logical outliers

```

1 # --- 1. SETUP AND STRATIFIED SPLIT ---
2 target_cols = ['pass_fail', 'final_grade', 'final_score']
3
4 df_unlabeled = df[df[target_cols].isnull().any(axis=1)].copy()
5 df_known = df[df[target_cols].notnull().all(axis=1)].copy()
6 df_train_orig, df_test = train_test_split(
7     df_known,
8     test_size=0.2,
9     random_state=42,
10    stratify=df_known['pass_fail']
11 )
12 print("Dataset Splits before pseudo labeling:")
13 print(f"Initial Training Set:      {len(df_train_orig)}")
14 print(f"Hold-out Test Set:         {len(df_test)}")
15 print(f"Unlabeled Set:               {len(df_unlabeled)}")
16 print(f"Total Dataset:               {len(df)}")

```

Listing 3: Train-Test Split

B Regression Models Implementations

B.1 Linear Regression

```

1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error, r2_score
3 import numpy as np
4
5 # Initialize the Linear Regression model
6 linear_model = LinearRegression()
7

```

```

8 # Fit the model to the scaled training data
9 linear_model.fit(X_train_scaled, y_train)
10
11 # Make predictions on the scaled test data
12 y_pred = linear_model.predict(X_test_scaled)
13
14 y_train_pred=linear_model.predict(X_train_scaled)
15
16 # Calculate Mean Squared Error
17 mse = mean_squared_error(y_test, y_pred)
18 rmse = np.sqrt(mse)
19 train_mse = mean_squared_error(y_train, y_train_pred)
20 train_rmse = np.sqrt(train_mse)
21 train_r2 = r2_score(y_train, y_train_pred)
22 test_r2=r2_score(y_test, y_pred)

```

Listing 4: Linear Regression Model

B.2 Lasso Regression

```

1 from sklearn.linear_model import LassoCV
2 from sklearn.metrics import mean_squared_error, r2_score
3
4 # Initialize LassoCV model
5 # n_alphas determines how many alphas to check. cv specifies the number of folds for
   cross-validation.
6 lasso_cv_model = LassoCV(alphas=None, cv=5, random_state=42, max_iter=10000)
7
8 # Fit the model to the scaled training data
9 lasso_cv_model.fit(X_train_scaled, y_train)
10
11 print(f"Best alpha found by LassoCV: {lasso_cv_model.alpha_:.4f}")
12
13 # Make predictions on the training data
14 y_train_pred_lasso = lasso_cv_model.predict(X_train_scaled)
15
16 # Calculate MSE for the training set
17 mse_train_lasso = mean_squared_error(y_train, y_train_pred_lasso)
18 r2_train_lasso = r2_score(y_train, y_train_pred_lasso)
19
20 # Make predictions on the test data
21 y_test_pred_lasso = lasso_cv_model.predict(X_test_scaled)
22
23 # Calculate MSE for the test set
24 mse_test_lasso = mean_squared_error(y_test, y_test_pred_lasso)
25 r2_test_lasso = r2_score(y_test, y_test_pred_lasso)
26
27 print(f"\nLassoCV Mean Squared Error (Training Set): {mse_train_lasso:.4f}")
28 print(f"LassoCV R-squared (Training Set): {r2_train_lasso:.4f}")
29 print(f"LassoCV Mean Squared Error (Test Set): {mse_test_lasso:.4f}")
30 print(f"LassoCV R-squared (Test Set): {r2_test_lasso:.4f}")

```

Listing 5: Lasso Regression Model

B.3 Support Vector Regression

```

1 from sklearn.model_selection import RandomizedSearchCV
2 from sklearn.svm import SVR

```

```

3 import numpy as np
4
5 # 1. Define a distribution rather than a fixed list
6 # This allows the search to try a wider variety of values
7 param_distributions = {
8     # Move into higher penalty zones
9     'C': [0.1,1,10,50, 100],
10
11     # Test finer influence ranges
12     'gamma': [0.01, 0.05, 0.1, 'scale'],
13
14     # Force the model to be more precise
15     'epsilon': [0.01, 0.05, 0.1,0.2],
16
17     'kernel': ['rbf']
18 }
19
20 # Increase n_iter to 20 to give it more chances to find the "sweet spot"
21 random_search = RandomizedSearchCV(
22     SVR(),
23     param_distributions=param_distributions,
24     n_iter=20,
25     cv=3,          # 3-fold is fine for 13k rows and faster
26     n_jobs=-1,
27     verbose=2,
28     scoring='neg_mean_squared_error',
29     random_state=42
30 )
31
32
33 print("Starting RandomizedSearchCV... this will be much faster.")
34 random_search.fit(X_train_scaled, y_train)
35
36 print(f"Best parameters: {random_search.best_params_}")
37 print(f"Best MSE (CV): {-random_search.best_score_:.4f}")
38
39
40 from sklearn.metrics import mean_squared_error
41
42 # 1. Get the best model from the search
43 best_svr = random_search.best_estimator_
44
45 # 2. Generate predictions for both sets
46 y_pred_train = best_svr.predict(X_train_scaled)
47 y_pred_test = best_svr.predict(X_test_scaled)
48
49 # 3. Calculate Errors
50 train_mse = mean_squared_error(y_train, y_pred_train)
51 test_mse = mean_squared_error(y_test, y_pred_test)
52 cv_mse = -random_search.best_score_ # Convert negative MSE back to positive
53
54 train_r2 = r2_score(y_train, y_pred_train)
55 test_r2 = r2_score(y_test, y_pred_test)

```

Listing 6: Support Vector Regression Model

C Multiclass Classification Model Implementations

C.1 XGBoost (Gradient Boosting)

```

1 from imblearn.over_sampling import SMOTE
2 from imblearn.pipeline import Pipeline
3 from sklearn.model_selection import RandomizedSearchCV
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.metrics import classification_report
6 from sklearn.utils.class_weight import compute_sample_weight
7 import xgboost as xgb
8 import numpy as np
9
10 # 1. Calculate Sample Weights for Multiclass
11 # This replaces scale_pos_weight for multiclass problems
12 weights = compute_sample_weight(class_weight='balanced', y=y_train_encoded)
13
14 # 2. Define the Pipeline
15 # SMOTE will now balance all classes to the size of the largest class
16 pipeline = Pipeline([
17     ('scaler', StandardScaler()),
18     ('smote', SMOTE(random_state=42)),
19     ('classifier', xgb.XGBClassifier(
20         objective='multi:softprob',
21         random_state=42
22     ))
23 ])
24
25 # 3. Hyperparameter Grid
26 param_dist = {
27     'classifier__n_estimators': [100, 200, 500, 1000],
28     'classifier__max_depth': [4, 6, 8],
29     'classifier__learning_rate': [0.05, 0.1],
30     'smote__k_neighbors': [3, 5]
31 }
32
33 # 4. Setup Randomized Search
34 # Use 'f1_macro' or 'f1_weighted' for multiclass imbalance
35 random_search = RandomizedSearchCV(
36     pipeline,
37     param_distributions=param_dist,
38     n_iter=10,
39     cv=3,
40     scoring='f1_macro',
41     random_state=42,
42     n_jobs=-1
43 )
44
45 # 5. Fit the model
46 # We pass the sample_weights to the 'classifier' step of the pipeline
47 random_search.fit(X_train, y_train_encoded)
48
49 best_model = random_search.best_estimator_

```

Listing 7: XGBoost Implementation with Imbalance Weighting

C.2 Random Forest Classifier

```

1 from sklearn.ensemble import RandomForestClassifier

```

```

2
3 # 1. Define the Pipeline
4 rf_pipeline = Pipeline([
5     ('scaler', StandardScaler()),
6     ('smote', SMOTE(random_state=42)),
7     ('classifier', RandomForestClassifier(class_weight='balanced', random_state=42))
8 ])
9
10 # 2. Hyperparameter Grid
11 rf_param_dist = {
12     'classifier__n_estimators': [100, 300, 500, 1000],
13     'classifier__max_depth': [None, 5, 10, 20],
14     'classifier__min_samples_split': [2, 5, 10],
15     'smote__k_neighbors': [3, 5]
16 }
17
18 # 3. Randomized Search
19 rf_search = RandomizedSearchCV(
20     rf_pipeline,
21     param_distributions=rf_param_dist,
22     n_iter=10,
23     cv=3,
24     scoring='f1_macro',
25     random_state=42,
26     n_jobs=-1
27 )
28
29 rf_search.fit(X_train, y_train_encoded)

```

Listing 8: Random Forest with Balanced Class Weights

C.3 Support Vector Classifier (SVC)

```

1 from sklearn.svm import SVC
2 from sklearn.utils.class_weight import compute_class_weight
3
4 # Calculate class weights dictionary for SVC 'class_weight' parameter
5 classes = np.unique(y_train_encoded)
6 weights = compute_class_weight(class_weight='balanced', classes=classes, y=
7     y_train_encoded)
8 class_weights_dict = dict(zip(classes, weights))
9
10 # 1. Define the Pipeline
11 svc_pipeline = Pipeline([
12     ('scaler', StandardScaler()),
13     ('smote', SMOTE(random_state=42)),
14     ('classifier', SVC(probability=True, class_weight=class_weights_dict))
15 ])
16
17 # 2. Hyperparameter Grid
18 # SVC parameters like C and gamma are critical for the decision boundary
19 svc_param_dist = {
20     'classifier__C': [0.1, 1, 10],
21     'classifier__gamma': ['scale', 'auto'],
22     'classifier__kernel': ['rbf'],
23     'smote__k_neighbors': [3, 5]
24 }
25
26 # 3. Randomized Search

```

```

26 svc_search = RandomizedSearchCV(
27     svc_pipeline,
28     param_distributions=svc_param_dist,
29     n_iter=8, # SVC is computationally expensive
30     cv=3,
31     scoring='f1_macro',
32     random_state=42,
33     n_jobs=-1
34 )
35
36 svc_search.fit(X_train, y_train_encoded)

```

Listing 9: Non-Linear SVC with RBF Kernel

D Binary Classification Models Implementation

```

1  # =====
2  # 1. Imports
3  # =====
4  import pandas as pd
5  import numpy as np
6  import matplotlib.pyplot as plt
7  import seaborn as sns
8
9  from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import LabelEncoder, StandardScaler
11 from sklearn.impute import SimpleImputer
12 from imblearn.over_sampling import SMOTENC
13
14 from sklearn.linear_model import LogisticRegression
15 from sklearn.neighbors import KNeighborsClassifier
16 from sklearn.svm import SVC
17 from sklearn.naive_bayes import GaussianNB
18 from sklearn.tree import DecisionTreeClassifier
19 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
20
21 from sklearn.metrics import (
22     accuracy_score, precision_score, recall_score, f1_score,
23     confusion_matrix, roc_curve, auc
24 )
25
26 # =====
27 # 2. Load dataset
28 # =====
29 df = pd.read_csv("Term_Project_Dataset_20K.csv")
30
31 # =====
32 # 3. Drop unnecessary columns
33 # =====
34 df = df.drop(columns=["final_score", "final_grade"])
35
36 # =====
37 # 4. Remove rows with missing target & encode target
38 # =====
39 df = df[df["pass_fail"].notna()]
40 X = df.drop(columns=["pass_fail"])
41
42 # Encode target: Fail = 0, Pass = 1

```



```

43 y = df["pass_fail"].map({"Fail": 0, "Pass": 1})
44
45 # =====
46 # 5. Encode categorical features
47 # =====
48 X_encoded = X.copy()
49 label_encoders = {}
50
51 categorical_cols = X_encoded.select_dtypes(include=["object", "category"]).columns
52 numeric_cols = X_encoded.select_dtypes(include=["int64", "float64"]).columns
53
54 for col in categorical_cols:
55     le = LabelEncoder()
56     X_encoded[col] = le.fit_transform(X_encoded[col].astype(str))
57     label_encoders[col] = le
58
59 # =====
60 # 6. Train-test split
61 # =====
62 X_train, X_test, y_train, y_test = train_test_split(
63     X_encoded, y,
64     test_size=0.2,
65     random_state=42,
66     stratify=y
67 )
68
69 # =====
70 # 7. Impute missing values
71 # =====
72 num_imputer = SimpleImputer(strategy="median")
73 cat_imputer = SimpleImputer(strategy="most_frequent")
74
75 X_train[numeric_cols] = num_imputer.fit_transform(X_train[numeric_cols])
76 X_test[numeric_cols] = num_imputer.transform(X_test[numeric_cols])
77
78 X_train[categorical_cols] = cat_imputer.fit_transform(X_train[categorical_cols])
79 X_test[categorical_cols] = cat_imputer.transform(X_test[categorical_cols])
80
81 # =====
82 # 8. Apply SMOTENC
83 # =====
84 categorical_features = [X_train.columns.get_loc(col) for col in categorical_cols]
85
86 smote = SMOTENC(categorical_features=categorical_features, random_state=42)
87 X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
88
89 # =====
90 # 9. Scaling
91 # =====
92 scaler = StandardScaler()
93 X_train_scaled = scaler.fit_transform(X_train_smote)
94 X_test_scaled = scaler.transform(X_test)
95
96 # =====
97 # 10. Classification Models
98 # =====
99 models = {
100     "Logistic Regression": LogisticRegression(max_iter=1000),
101     "KNN": KNeighborsClassifier(n_neighbors=5),
102     "SVM": SVC(probability=True),

```

```

103     "Naive Bayes": GaussianNB(),
104     "Decision Tree": DecisionTreeClassifier(random_state=42),
105     "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
106     "Gradient Boosting": GradientBoostingClassifier(random_state=42)
107 }
108
109 # =====
110 # 11. Classification Evaluation Function
111 # =====
112 def evaluate_classification(model, X_test, y_test, name):
113     y_pred = model.predict(X_test)
114     y_prob = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba")
115         else None
116
117     print(f"\n===== {name} =====")
118     print("Accuracy :", accuracy_score(y_test, y_pred))
119     print("Precision:", precision_score(y_test, y_pred))
120     print("Recall   :", recall_score(y_test, y_pred))
121     print("F1-score :", f1_score(y_test, y_pred))
122
123 # =====
124 # 12. Train & Evaluate Classification Models
125 # =====
126 for name, model in models.items():
127     if name in ["Decision Tree", "Random Forest", "Gradient Boosting"]:
128         model.fit(X_train_smote, y_train_smote)
129         evaluate_classification(model, X_test, y_test, name)
130     else:
131         model.fit(X_train_scaled, y_train_smote)
132         evaluate_classification(model, X_test_scaled, y_test, name)
133
134 # =====
135 # 13. Permutation Importance
136 # =====
137 from sklearn.inspection import permutation_importance
138
139 r = permutation_importance(
140     model,
141     X_test_scaled if not isinstance(model, (DecisionTreeClassifier,
142                                             RandomForestClassifier, GradientBoostingClassifier)) else X_test,
143     y_test,
144     n_repeats=10,
145     random_state=42
146 )
147
148 perm_df = pd.DataFrame({
149     "Feature": X_train.columns,
150     "Importance": r.importances_mean
151 }).sort_values(by="Importance", ascending=False)
152
153 print(perm_df.head(10))

```

Listing 10: Supervised Classification Pipeline and Feature Importance

E Semi-Supervised Approach

E.1 Clustering and Pseudo-Labeling Pipeline

```

1  # --- 2. CLUSTERING FOR PSEUDO-LABELING ---
2  scaler_pre = StandardScaler()
3  X_train_pre = df_train_orig.drop(columns=target_cols)
4  X_unlabeled_pre = df_unlabeled.drop(columns=target_cols)
5
6  X_train_scaled_pre = scaler_pre.fit_transform(X_train_pre)
7  X_unlabeled_scaled_pre = scaler_pre.transform(X_unlabeled_pre)
8
9  k = 30
10 kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
11 df_train_orig['cluster'] = kmeans.fit_predict(X_train_scaled_pre)
12 df_unlabeled['cluster'] = kmeans.predict(X_unlabeled_scaled_pre)
13
14 # --- 3. CALCULATE CLUSTER STATISTICS ---
15 cluster_stats = {}
16 for c_id in range(k):
17     cluster_subset = df_train_orig[df_train_orig['cluster'] == c_id]
18     if not cluster_subset.empty:
19         mode_bin = cluster_subset['pass_fail'].mode()[0]
20         cluster_stats[c_id] = {
21             'binary': mode_bin,
22             'multi': cluster_subset['final_grade'].mode()[0],
23             'reg': cluster_subset['final_score'].mean(),
24             'purity': (cluster_subset['pass_fail'] == mode_bin).mean()
25         }
26
27 # --- 4. APPLY PSEUDO-LABELS ---
28 def apply_pseudo_labels(row):
29     c_id = row['cluster']
30     if c_id in cluster_stats and cluster_stats[c_id]['purity'] >= 0.95:
31         return pd.Series([cluster_stats[c_id]['binary'], cluster_stats[c_id]['multi'],
32                             cluster_stats[c_id]['reg']])
33     return pd.Series([np.nan, np.nan, np.nan])
34
35 df_unlabeled[target_cols] = df_unlabeled.apply(apply_pseudo_labels, axis=1)
36 df_pseudo_labeled = df_unlabeled.dropna(subset=target_cols).copy()
37
38 print("After Pseudo-Labeling:")
39 print(f"Pseudo-Labeled Set:      {len(df_pseudo_labeled)}")
40 print(f"Total Training Set:      {len(df_train_orig) + len(df_pseudo_labeled)}")
41 print(f"Total Dataset:            {len(df_train_orig) + len(df_pseudo_labeled) + len(df_test)}")
42
43 print(f"Pseudo-Labeling Added:    {len(df_pseudo_labeled)} samples")
44 # FINAL CONCATENATION
45 df_final_train = pd.concat([df_train_orig, df_pseudo_labeled], axis=0).reset_index(drop=True)
46 print(f"Final Training Set Size: {len(df_final_train)}")
47 # --- 5. FINAL FEATURE PREPARATION ---
48 features = [col for col in df_final_train.columns if col not in target_cols + ['cluster']]
49 scaler_mid = StandardScaler()
50 X_train_final_scaled = scaler_mid.fit_transform(df_final_train[features])
51 X_test_final_scaled = scaler_mid.transform(df_test[features])

```

Listing 11: Inductive clustering for label propagation